



CPU usage, one program at a time - an unfortunate waste of its time

The first answer was *multiprogramming*. Many programs would be loaded into memory, and one would run first. Now, if this program was waiting for a device (like a drive) to process its data, valuable CPU time would be wasted. Instead, the context (see §1.3.1, *Processes and Threads*) of the program would be stored in a temporary location, and a different program would be allowed to use the CPU. This way, more work could get done, and faster.

The multiprogramming approach soon gave way to a 'time-share' approach. The idea was that each program should 'give back' resources to the OS every so often, so that these resources could now be given to different programs. This was called *Co-operative Multitasking*. Developers would write programs that would save their context after running for a fixed time and hand control over to other programs. While this sped things along quite well, there were a couple of very serious flaws. If the programmer had made a mistake, the program might well end up hogging all the system's resources. At best, it would end up being the only useful program; at worst, it could crash and bring the whole system down with it.

Today's OSes use *Multithreading*—they prefer to switch between threads rather than whole processes. Context-switching for threads is faster because they don't have their own memory context—they run in the context of the process that created them. Rather than leaving it up to processes to multitask among themselves, the OS kernel takes charge and assigns them 'time-slots', within which they can use the system's resources. After their time has expired, the OS saves their context, loads the context of another process, and resumes execution from there. This is called *Pre-emptive Multitasking*.